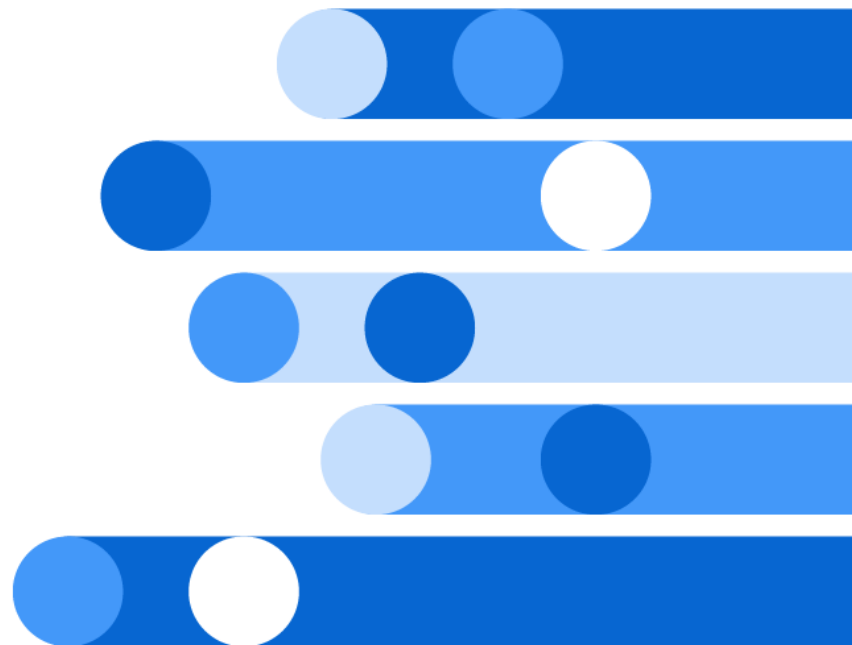


Generic

NMF



This document is an individual chapter from *Generic*.

The correct bibliographic citation for this manual is as follows: SAS Institute Inc. 2021. *Generic*. Cary, NC: SAS Institute Inc.

Generic

Copyright © 2021, SAS Institute Inc., Cary, NC, USA

All Rights Reserved. Produced in the United States of America.

For a hard-copy book: No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, or otherwise, without the prior written permission of the publisher, SAS Institute Inc.

For a web download or e-book: Your use of this publication shall be governed by the terms established by the vendor at the time you acquire this publication.

The scanning, uploading, and distribution of this book via the internet or any other means without the permission of the publisher is illegal and punishable by law. Please purchase only authorized electronic editions and do not participate in or encourage electronic piracy of copyrighted materials. Your support of others' rights is appreciated.

May 2021

SAS® and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.

Other brand and product names are trademarks of their respective companies.

SAS software may be provided with certain third-party software, including but not limited to open source software, which is licensed under its applicable third-party software license agreement. For license information about third-party software distributed with SAS software, refer to [Third-Party Software Reference | SAS Support](#).

NMF Call

CALL NMF(*W*, *H*, *A*, *k* <, *method*> <, *opt*> <, *W0*> <, *H0*>);

This subroutine is supported by the IML procedure and the **iml** action.

The NMF subroutine implements nonnegative matrix factorization (NMF). The NMF algorithm returns a low-rank approximation of a nonnegative matrix. Specifically, for a nonnegative $m \times n$ data matrix *A* and an integer *k* ($1 \leq k \leq n$), the NMF subroutine computes an $m \times k$ nonnegative matrix, W_k , and a $k \times n$ nonnegative matrix, H_k , such that

$$A \approx W_k H_k$$

The subroutine returns the matrix W_k in the argument *W* and returns the matrix H_k in the argument *H*. The product minimizes the Frobenius norm $\|A - W_k H_k\|_F$ subject to the constraints that $W_k \geq 0$ and $H_k \geq 0$.

The NMF optimization requires the initial guesses W_0 and H_0 for the product matrices. The NMF subroutine offers three different methods of providing initial guesses.

The two-factor approximation $A \approx W_k H_k$ is not unique. Let *D* be an invertible matrix. Then $A = (W_k D)(D^{-1} H_k)$, so $W_k D$ and $D^{-1} H_k$ are possible NMF factors. The NMF subroutine standardizes the output so that the columns of *W* are sorted by their norms and the rows of *H* have unit length.

The required input arguments to the NMF subroutine are as follows:

- A* is an $m \times n$ nonnegative numeric matrix. The matrix *A* cannot contain missing values.
- k* is a positive integer that specifies the approximation rank, where $1 \leq k \leq n$.

You can also specify the following optional arguments:

method is a string that specifies the factorization algorithm. You can choose the following algorithms:

- "ALS" specifies the alternating least squares algorithm (Paatero 1997, 1999). This is the default method.
- "MUP" specifies the multiplicative updates algorithm (Lee and Seung 2000).
- "APG" specifies the alternating proximal gradient algorithm (Guan et al. 2012).

opt is a numeric vector that has up to five elements. Each element of *opt* controls certain features of the NMF subroutine. You can use the OPT keyword to specify the vector *opt*. The default value is used if an element is omitted or is a missing value. In the following list, assume that the *opt* vector is the vector {verbose, maxIters, maxTry, normTol, xTol}:

- verbose* (**opt [1]**) specifies the amount of output to display. Valid values are 0, 1, or 2. The default value is 0, which displays no information. A value of 1 displays the information for only the last iteration step. A value of 2 displays information for every iteration step.
- maxIters* (**opt [2]**) specifies the maximum number of iterations for each initial guess for W_0 and H_0 . For the ALS method, the default value is 200. For other methods, the default value is 20,000.

- maxTry* (opt [3]) specifies the maximum number of random initial guesses for each W_0 and H_0 . The default value is 10. The NMF algorithm iterates for each initial guess. The subroutine returns the solution that has the smallest error, according to the convergence criterion.
- normTol* (opt [4]) specifies the Frobenius norm criterion for convergence. If the relative change in the Frobenius norm is less than the *normTol* value in two consecutive iterations, the computations stop for the current initial guess. The default value is 10^{-5} .
- xTol* (opt [5]) specifies a criterion for the maximum relative elementwise difference between A and WH in each iteration. If the relative maximum difference is below the *xTol* value, the iterations stop. The default value is 10^{-5} . The algorithm stops when either the *normTol* criterion or the *xTol* criterion is satisfied.
- W0* is either a string or an $m \times k$ numeric matrix. Valid string values are "SVD" and "RAND". If you choose "SVD", the subroutine performs a singular value decomposition of A and uses the absolute values of the SVD factors to initialize $W0$ and $H0$. In this case, the *H0* argument is ignored. If you specify "RAND", elements of $W0$ are drawn randomly from the uniform distribution on $U(0, 1)$. If you provide a nonnegative $m \times k$ matrix, it is used as an initial value for W .
- H0* is either a string or a $k \times n$ numeric matrix. Valid string values are "SVD" and "RAND". If you choose "SVD", the subroutine performs a singular value decomposition of A and uses the absolute values of the SVD factors to initialize $W0$ and $H0$. In this case, the *W0* argument is ignored. If you specify "RAND", elements of $H0$ are drawn randomly from the uniform distribution on $U(0, 1)$. If you provide a nonnegative $k \times n$ matrix, it is used as an initial value for H .

The NMF call returns the following matrices:

- W is an $m \times k$ numeric matrix. Columns are ordered from left to right according to their Frobenius norm.
- H is a $k \times n$ numeric matrix. H is scaled so that the sum of squares of each row is unity.

If you omit the *W0* and *H0* options, the NMF subroutine uses the SVD method to initialize W and H when the input matrix has fewer than 1,000 columns. Otherwise, it uses the "RAND" option. In most cases, the SVD initialization converges to a solution faster but uses more memory.

The following example shows how to use the NMF subroutine to compute the rank-2 approximation of the *Virginica* species of Fisher's Iris data. First, the numeric variables of the Iris data table are loaded into the matrix, X . The NMF call then computes W and H such that $X \approx W_k H_k$.

```
load module = _all_;    /* Load the NMF subroutine*/

/* Apply NMF to Virginica species of Fisher's Iris data */
use sashelp.iris where (Species='Virginica');
  read all var _NUM_ into X[c=varNames];
close;

print (X[1:5,])[c=varNames L="Virginica Species (First five observations)"];

call randseed(1234, 1);          /* Set random number seed for reproducibility */
names = 'NMF 1':'NMF 2';
```

```

call nmf(W, H, X, ncol(names)); /* Rank-2 nonnegative factorization */

print (W[1:5,])[c=names l='W (First five rows)',
      H[c=varNames r=names l='H'];

Y = W*H; /* Rank-2 approximation of X */
print (Y[1:5,])[c=varNames F=4.1 l="NMF Rank-2 Approximation to First Five Observations"];

RMS = norm(X-Y, "Frobenius") / sqrt(ncol(X)*nrow(X));
print RMS[F=4.2 l="Root Mean Square Error of NMF Approximation"];

```

Figure 0.1 Rank-2 Approximation of Virginica Species in Fisher's Iris Data

Virginica Species (First five observations)				
SepalLength	SepalWidth	PetalLength	PetalWidth	
64	28	56	22	
67	31	56	24	
63	28	51	15	
69	31	51	23	
65	30	52	20	

W (First five rows)		
NMF1		NMF2
89.489324		7.0464147
91.823888		9.9290242
85.546685		3.9501565
90.023955		9.9696388
87.624048		8.0032532

H				
	SepalLength	SepalWidth	PetalLength	PetalWidth
NMF1	0.723397	0.2684271	0.6115087	0.1752162
NMF2	0.032492	0.7764129	0.0027986	0.6293803

NMF Rank-2 Approximation to First Five Observations				
SepalLength	SepalWidth	PetalLength	PetalWidth	
65.0	29.5	54.7	20.1	
66.7	32.4	56.2	22.3	
62.0	26.0	52.3	17.5	
65.4	31.9	55.1	22.0	
63.6	29.7	53.6	20.4	

Root Mean Square Error of NMF Approximation	
1.46	

You can examine the accuracy of the approximation by comparing the difference between Y , which is the product of W and H , and the original data matrix X . The matrix Y is a rank-2 approximation of X . To quantify the overall quality of the approximation, you can compute the root mean square (RMS) error. The RMS that you obtain is an indicator of how closely W and H approximate A .

References

- Guan, N., Tao, D., Luo, Z., and Yuan, B. (2012). “NeNMF: An Optimal Gradient Method for Nonnegative Matrix Factorization.” *IEEE Transactions on Signal Processing* 60:2882–2898.
- Lee, D. D., and Seung, H. S. (2000). “Algorithms for Non-negative Matrix Factorization.” In *Advances in Neural Information Processing Systems 13 (NIPS 2000)*, vol. 13, 556–562. Cambridge, MA: MIT Press. https://proceedings.neurips.cc/paper_files/paper/2000/file/f9d1152547c0bde01830b7e8bd60024c-Paper.pdf.
- Paatero, P. (1997). “Least Squares Formulation of Robust Non-negative Factor Analysis.” *Chemometrics and Intelligent Laboratory Systems* 37:23–35.
- Paatero, P. (1999). “The Multilinear Engine: A Table-Driven, Least Squares Program for Solving Multilinear Problems, Including the n -way Parallel Factor Analysis Model.” *Journal of Computational and Graphical Statistics* 8:854–888.